formalmind

Security Assessment & Formal Verification

asyncswap/contracts

Formal Mind

Version bf08300, 2026-10-21: Security assessment

Table of Contents

1. Introduction . . . **1**
1.1. About Formal Mind . . . 1
1.2. About This Report . . . 1
1.3. Disclaimer . . . 1
2. Project Summary . . . **2**
2.1. Project Scope . . . 2
2.2. Project Overview . . . 2
2.3. Findings Summary . . . 3
2.4. Severity Matrix . . . 3
3. Detailed Findings . . . **4**
4. Formal Verification . . . **5**
4.1. Assumptions and Simplifications . . . 5
4.2. Verification Notations . . . 5
4.3. Coverage . . . 5
4.4. Formal Verification Properties . . . 6
4.4.1. Router . . . 6
4.4.2. BaseHook . . . 10
4.4.3. AsyncSwap . . . 18
4.4.4. NativeClaims . . . 40
4.4.5. FeeVault . . . 42
Appendix A: Function Coverage . . . **45**

1. Introduction

1.1. About Formal Mind

Formal Mind is a formal audit workspace for Solidity. A local swarm of agents reads the contracts through the Solidity compiler's AST, proves their functions against a Lean 4 model generated from it, and files every bug it finds with a Foundry proof of concept. Its reviewer checks each proof before it counts. Learn more at formal.team.

1.2. About This Report

This report presents Formal Mind's security assessment of asyncswap/contracts at commit bf08300: the bugs its swarm found, each with a proof of concept; the Lean proofs of the code and how much of the code they cover. Every code reference links to the audited commit, and every fix to the pull request or commit that made it.

The report is generated from `formal-mind.audit.json` (the `formalmind.audit/v1` format), which is the source of truth; edit that file and generate the report again to change it.

1.3. Disclaimer

This assessment is a snapshot of the code at the commit above and does not guarantee against a hack. Proofs hold for the Lean model formal-ast generates, under the assumptions stated in Assumptions and Simplifications. Code the model leaves opaque, the compiler, and contracts outside the scope are not verified. Any change to the code needs a new assessment.

2. Project Summary

2.1. Project Scope

Project	Repository	Commit	Platform
asyncswap/contracts	asyncswap/contracts	bf08300	EVM / Solidity 0.8.26 Lean 4.35.0

2.2. Project Overview

Formal Mind's swarm verified asyncswap/contracts against a Lean 4 model generated from the Solidity compiler's AST: 126 of 197 functions in scope are proved, with 42 reviewed properties. It filed 0 findings, each with a proof-of-concept test.

The contracts in scope are the ones the repository's Foundry scripts under `script/` deploy or call, with the code they reach.

The following files are in scope:

Table 1. Files in scope

File	Lines	Code
src/AsyncSwap.sol	612	430
src/BaseHook.sol	215	171
src/FeeVault.sol	46	34
src/ImmutableState.sol	23	13
src/NativeClaims.sol	44	32
src/Router.sol	360	248
src/libraries/AsyncFiller.sol	440	351
src/libraries/AsyncMatching.sol	439	378
src/libraries/CurrencySettler.sol	38	26
src/libraries/OrderSignatures.sol	136	104
src/libraries/RadixPriceTree.sol	174	145

File	Lines	Code
src/types/AsyncOrder.sol	61	24
Total (12 files)	**2588**	**1956**

2.3. Findings Summary

Severity	Found	Fixed	Acknowledged	Open
Critical	0	0	0	0
High	0	0	0	0
Medium	0	0	0	0
Low	0	0	0	0
Informational	0	0	0	0
Total	**0**	**0**	**0**	**0**

2.4. Severity Matrix

Severity combines what a bug costs when exploited (impact) with how likely it is to be (likelihood).

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	**Medium**	**High**

Likelihood runs left to right.

3. Detailed Findings

Every property the swarm checked at commit bf08300 holds. Its results are the verified properties in Formal Verification.

4. Formal Verification

4.1. Assumptions and Simplifications

1. formal-ast compiles the contracts with solc 0.8.26 and translates solc's AST into a Lean 4.35.0 model; the theorems are about that model.
2. Functions formal-ast can't translate stay opaque (not modeled): a theorem then holds for every behaviour of them, and says nothing about their own code.
3. External calls are parameters of the model, so a theorem holds whatever the called contract does unless it states otherwise.
4. A function counts as proved when a theorem without `sorry` names its Lean definition. The swarm's own proofs also pass `lake build` with no `sorry`, `admit`, `axiom` or `native_decide`, and are approved by its reviewer.

4.2. Verification Notations

Proved	Verified: a theorem without `sorry` names the function's Lean definition, so the property holds for every state and input of the model, under the assumptions above.
Has sorry	Theorems name it, but each still has a `sorry`: the proof isn't finished.
In Lean	Modeled in Lean, but no theorem about it yet.
Not modeled	Left opaque by formal-ast: theorems hold for every behaviour of it (the reason is in the appendix).

4.3. Coverage

126 of **197** functions are proved (64.0%); 169 are in the Lean model and 28 are not modeled. The repository has 876 theorems.

Contract	Functions	In Lean	Proved	Not modeled	Proved
AsyncSwap	143	128	95	15	66.4%
FeeVault	4	3	3	1	75.0%
NativeClaims	4	3	3	1	75.0%

Contract	Functions	In Lean	Proved	Not modeled	Proved
Router	46	35	25	11	54.3%
Total	**197**	**169**	**126**	**28**	**64.0%**

Every function's status is in Function Coverage.

4.4. Formal Verification Properties

Each property is a function the swarm's prover proved against the model and its reviewer approved: what the proof establishes, in the prover's words, and the theorems that prove it, each linked where it is.

4.4.1. Router

P-10. `Router.matchBook`		
Status: **Verified**	Function: src/Router.sol#L228	Proof: formal/Refinement/Swarm/RouterMatchBook.lean

- `matchBook_eq`: `Router.matchBook ext key m env w` does exactly one thing. It calls `ext.call_IPoolManager_unlock ext.POOLMANAGER (matchBookData key m)` and throws away the result. `matchBookData` is the encoded callback: action 4 (MatchBook), the five pool key fields, then seven zero words for the placeholder order, then `maxMatches`.
- `matchBook_revert_iff`: it reverts only when the `unlock` call reverts, and with the same error.
- `matchBook_ok_iff`: it succeeds with final state `w'` exactly when `unlock` succeeds and ends in `w'`. The Router adds no state change of its own.

Theorem	Status	Where
`matchBook_eq`	Verified	formal/Refinement/Swarm/RouterMatchBook.lean#L27
`matchBook_revert_iff`	Verified	formal/Refinement/Swarm/RouterMatchBook.lean#L34
`matchBook_ok_iff`	Verified	formal/Refinement/Swarm/RouterMatchBook.lean#L43

P-11. `Router.fillFromPoolLiquidity`		
Status: **Verified**	Function: src/Router.sol#L238	Proof: formal/Refinement/Swarm/RouterFillFromPoolLiquidity.lean

- `fillFromPoolLiquidity_eq`: the function only does two things. First it writes two transient (per-transaction) slots: `USER_LOCATION` gets the caller and `ASYNC_FILLER_LOCATION` gets the Router's own address. Then it makes one call, `POOLMANAGER.unlock(callbackData 5 order 0)`, and ignores the result. That call data is the encoded `SwapCallback` with action 5 (`PoolLiquidityFill`), the order and `maxMatches = 0`.
- `fillFromPoolLiquidity_revert_iff` (guards): the function has no checks of its own, so it reverts exactly when `unlock` reverts, with the same error.
- `fillFromPoolLiquidity_ok_iff` (footprint): on success, the end state is whatever `unlock` leaves, starting from the state after the two transient writes. The Router writes no persistent storage itself.
- `fillFromPoolLiquidityPre_get`: assuming `ext.get` reads back what `ext.set` wrote, those two writes change only the `transient` field.

These proofs cover the behaviour above; they don't establish that the function is safe overall. The function has no access check, so anyone can call it. Whether that is safe depends on how the `unlockCallback` side handles the `PoolLiquidityFill` action, which this item doesn't cover.

Theorem	Status	Where
`fillFromPoolLiquidity_eq`	Verified	formal/Refinement/Swarm/RouterFillFromPoolLiquidity.lean#L28
`fillFromPoolLiquidity_revert_iff`	Verified	formal/Refinement/Swarm/RouterFillFromPoolLiquidity.lean#L37
`fillFromPoolLiquidity_ok_iff`	Verified	formal/Refinement/Swarm/RouterFillFromPoolLiquidity.lean#L48

P-19. `Router.swapWithSig`		
Status: **Verified**	Function: src/Router.sol#L178	Proof: formal/Refinement/Swarm/RouterSwapWithSig.lean

I proved plan item 19: `Router.swapWithSig` behaves exactly as its slice says. The proofs are in `formal/Refinement/Swarm/RouterSwapWithSig.lean`, and `lake build Refinement.Swarm.RouterSwapWithSig` passes with no `sorry`, `admit`, `native_decide` or new axioms.

Theorem	Status	Where
`swapWithSig_native`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L47
`swapWithSig_noGrant`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L55
`swapWithSig_alreadyExecutor`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L64
`swapWithSig_isExecutor_revert`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L75
`swapWithSig_grant`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L88
`swapWithSig_ok_input`	Verified	formal/Refinement/Swarm/RouterSwapWithSig.lean#L107

P-21. `Router.cancelWithSig`

Status: **Verified**	Function: src/Router.sol#L198	Proof: formal/Refinement/Swarm/RouterCancelWithSig.lean, formal/.tmp_unused

What the theorems show: * **Guards, in order: An expired deadline reverts with** `SignatureExpired`**.** A used `(owner, nonce)` pair reverts with `NonceAlreadyUsed`. **If the** `HOOK.asyncOrderInfo` **view reverts, the whole call reverts.** A signature that doesn't check out for `order.owner` reverts with `InvalidSignature`. * **Success:** the call ends in one `POOLMANAGER.unlock(SwapCallback(Cancel, order, 0))`. * **Inversion (**`cancelWithSig_ok_inv`**):** any call that succeeds passed every guard. That includes a valid signature from `order.owner` over the digest binding the owner, both tokens, the hook's live `amountIn`, the order id, the nonce and the deadline. * **Footprint (**`cancelWithSig_ok_state`**):** the only storage the Router writes before handing off to `unlock` is `cancelNonceUsed[order.owner][nonce] = true`. It writes no transient slot and moves no funds itself. * **No replay:** using the same `(owner, nonce)` again reverts, whether re-entered from inside `unlock` (`cancelWithSig_reentry_reverts`) or called later (`cancelWithSig_no_replay`).

For your concern: anyone can submit a cancel for someone else's order, but it only goes through with the owner's signature, and each signature works once.

Assumptions the theorems take as hypotheses: * The model leaves `toTypedDataHash` and the ERC-1271 signature check as opaque calls, so the footprint and no-replay results assume they don't change

storage. Both are pure or static calls in Solidity. * The after-the-call no-replay result also assumes `unlock` never sets a used cancel nonce back to `false`. The Router has no code that does.

Theorem	Status	Where
`cancelWithSig_expired`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L107
`cancelWithSig_nonce_used`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L112
`cancelWithSig_info_revert`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L118
`cancelWithSig_eq_prefix`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L127
`cancelWithSig_invalid_sig`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L150
`cancelWithSig_ok`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L163
`cancelWithSig_ok_inv`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L178
`cancelWithSig_ok_state`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L224
`cancelWithSig_reentry_reverts`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L254
`cancelWithSig_no_replay`	Verified	formal/Refinement/Swarm/RouterCancelWithSig.lean#L266

P-45. `Router.DOMAIN_SEPARATOR`

Status: **Verified**	Function: src/Router.sol#L213	Proof: formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean

Each theorem names `Router.DOMAIN_SEPARATOR` fully qualified: * `DOMAIN_SEPARATOR_spec`: the call always succeeds and leaves storage as it was. It returns the EIP-712 domain separator: Cached value: **returned when** `address(this)` **and** `block.chainid` **match the values cached at deployment. Rebuilt value:** otherwise, `keccak(TYPE_HASH, _hashedName, _hashedVersion, chainid, self)`. * `DOMAIN_SEPARATOR_never_reverts`: the function has no guards, so it never reverts. *

`DOMAIN_SEPARATOR_preserves`: on success it writes nothing, and the value returned is exactly the domain separator above.

Theorem	Status	Where
`DOMAIN_SEPARATOR_spec`	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L29
`DOMAIN_SEPARATOR_never_reverts`	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L40
`DOMAIN_SEPARATOR_preserves`	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L45

4.4.2. BaseHook

P-12. `BaseHook.beforeInitialize`		
Status: **Verified**	Function: src/BaseHook.sol#L35	Proof: formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean, formal/tmp_snippet.txt, formal/Scratch_BeforeInit.lean

What is proved: * **Who can call it:** any caller other than `poolManager` gets `NotPoolManager`. * **Pool must include the numeraire:** if neither currency is the numeraire, the call reverts with `NumeraireRequired`. * **Pool must use a dynamic fee:** if `key.fee ≠ DYNAMIC_FEE_FLAG`, the call reverts with `DynamicFeeRequired`. * **Exact success result** (`beforeInitialize_run_ok`): when all three checks pass, it returns `0xdc98354e` (the `beforeInitialize` selector). The resulting world is the three writes the generated code makes, in order. This theorem needs no extra assumptions. * **Succeeds exactly when the three checks pass** (`beforeInitialize_succeeds_iff`). A companion theorem shows that any success returns the selector and leaves the stated storage. * **What it changes** (`beforeInitialize_frame`): only the `poolManager`, `numeraire` and `feeVault` fields of `asyncOrders[key.toId()]` change. Every other pool and every other state variable is untouched.

The storage-level results assume only that reading the hook's storage back returns what was just written (`ext.get (ext.set w s) = s`). There are no external calls.

Housekeeping: I left two empty files, `formal/Scratch_BeforeInit.lean` and `formal/tmp_snippet.txt`. I couldn't delete them with the tools I have, so please remove them.

Next: the order create, fill and settle paths are where your self-fill concern actually matters. I'd prove there that no order can take out more than its maker put in.

Theorem	Status	Where
`beforeInitialize_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L52
`beforeInitialize_numeraireRequired`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L58
`beforeInitialize_dynamicFeeRequired`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L67
`beforeInitialize_run_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L78
`beforeInitialize_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L97
`beforeInitialize_ok_inv`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L105
`beforeInitialize_succeeds_iff`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L124
`beforeInitialize_frame`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeInitialize.lean#L145

P-30. `BaseHook.beforeSwap`		
Status: **Verified**	Function: src/BaseHook.sol#L143	Proof: formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean, formal/Refinement/Swarm/.tmp, formal/Scratch.lean

The generated def only covers part of the function. `_createOrder` is left as an unexplained black box in the model, so 4 of the slice's 9 guards can't be proved here: `isExecutor`, the zero-amount check, the priority-fee rules and the node-id overflow. Those need a proof about `_createOrder` itself.

What's proved: * Reverts, in order: `NotPoolManager` **unless the caller is the pool manager.** `ExactInputOnly` when `amountSpecified > 0`. **A** `0x11` **overflow panic on negating** `type(int256).min`. `abi.decode` when the hook data is shorter than 128 bytes or the user or executor word isn't a valid address. **An executor-mismatch revert when** `sender` **isn't** `hookData.executor`**. The model gives it an empty reason string rather than** `NotAuthorizedExecutor`**. *** Success: `beforeSwap_run_ok` **shows that once those checks pass, the call is exactly** `poolManager.mint(this, specified, amountTaken)`**, then** `_createOrder(poolId, zeroForOne, amountTaken, decoded hookData, specified == numeraire)`,

then a return. `beforeSwap_ok_inv` **and** `beforeSwap_succeeds_iff` **give the converse: it succeeds exactly when those checks hold and both calls succeed. *** Return value: **always** `(0x575e24b4, delta, 0)`**. The escrowed amount is exactly** `-amountSpecified`**. When that amount is below 2^127, the delta is exactly** `amountTaken × 2^128`**. *** Storage:** the function writes nothing itself. Every change to `asyncOrders` or `priceBooks` comes from the `mint` call or `_createOrder` (`beforeSwap_frame`).

The model also drops event emits, so it misses one revert: in Solidity, `toInt128` inside the `HookSwap` emit reverts when the amount is 2^127 or more. The real contract is stricter there, so that's not a bug, and the file's header says so.

Theorem	Status	Where
`beforeSwap_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L78
`beforeSwap_exactInputOnly`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L84
`beforeSwap_negOverflow`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L91
`beforeSwap_badDecode`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L101
`beforeSwap_notExecutor`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L143
`beforeSwap_run_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L161
`beforeSwap_ok_inv`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L178
`beforeSwap_ok_returns`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L200
`beforeSwap_succeeds_iff`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L213
`beforeSwap_frame`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L227
`beforeSwap_ok_delta`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeSwap.lean#L288

P-46. `BaseHook.afterInitialize`

Status: **Verified**	Function: src/BaseHook.sol#L48	Proof: formal/Refinement/Swarm/AsyncSwapAfterInitialize.lean

AsyncSwap does not override `_afterInitialize`, so this function always reverts and never writes storage, returns a value or makes an external call. That is expected for a hook AsyncSwap doesn't implement, so I don't treat it as a bug. Each theorem names `BaseHook.afterInitialize`: * `afterInitialize_notPoolManager`: if the caller isn't the pool manager, it reverts with `NotPoolManager`. * `afterInitialize_hookNotImplemented`: if the caller is the pool manager, it reverts with `HookNotImplemented`. * `afterInitialize_spec`: the exact result: it always reverts, and the caller decides which error. * `afterInitialize_never_ok`: it never succeeds.

Theorem	Status	Where
`afterInitialize_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterInitialize.lean#L20
`afterInitialize_hookNotImplemented`	Verified	formal/Refinement/Swarm/AsyncSwapAfterInitialize.lean#L26
`afterInitialize_spec`	Verified	formal/Refinement/Swarm/AsyncSwapAfterInitialize.lean#L33
`afterInitialize_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapAfterInitialize.lean#L45

P-47. `BaseHook.beforeAddLiquidity`

Status: **Verified**	Function: src/BaseHook.sol#L61	Proof: formal/Refinement/Swarm/AsyncSwapBeforeAddLiquidity.lean

The four theorems say: * `beforeAddLiquidity_notPoolManager`: if the caller is not the pool manager, the call reverts with `NotPoolManager`. * `beforeAddLiquidity_ok`: if the caller is the pool manager, it returns `0x259982e5` (the `beforeAddLiquidity` selector) and leaves storage unchanged. * `beforeAddLiquidity_spec`: combines the two: it reverts exactly when the caller is not the pool manager, and otherwise returns the selector. * `beforeAddLiquidity_frame`: any successful call returns the selector, leaves storage untouched, and must have come from the pool manager.

Theorem	Status	Where
`beforeAddLiquidity_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeAddLiquidity.lean#L19

Theorem	Status	Where
`beforeAddLiquidity_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeAddLiquidity.lean#L26
`beforeAddLiquidity_spec`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeAddLiquidity.lean#L35
`beforeAddLiquidity_frame`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeAddLiquidity.lean#L47

P-48. `BaseHook.beforeRemoveLiquidity`

Status: **Verified**	Function: src/BaseHook.sol#L79	Proof: formal/Refinement/Swarm/AsyncSwapBeforeRemoveLiquidity.lean

The function can never succeed in AsyncSwap. It keeps the base `_beforeRemoveLiquidity`, which always reverts with `HookNotImplemented`. The theorems, each naming `BaseHook.beforeRemoveLiquidity` in full: * `beforeRemoveLiquidity_notPoolManager`: if the caller isn't the pool manager, it reverts with `NotPoolManager`. * `beforeRemoveLiquidity_hookNotImplemented`: if the caller is the pool manager, it reverts with `HookNotImplemented`. * `beforeRemoveLiquidity_spec`: for any caller, the result is exactly one of those two reverts, depending on who calls. * `beforeRemoveLiquidity_never_ok`: there is no successful run, so it writes no storage, returns nothing and makes no external calls.

This isn't a bug. The hook permissions say AsyncSwap doesn't use `beforeRemoveLiquidity` (the flag is `false`), so no report was needed. The proofs follow the same pattern as the existing `AsyncSwapAfterInitialize` file.

Theorem	Status	Where
`beforeRemoveLiquidity_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeRemoveLiquidity.lean#L20
`beforeRemoveLiquidity_hookNotImplemented`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeRemoveLiquidity.lean#L28
`beforeRemoveLiquidity_spec`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeRemoveLiquidity.lean#L37
`beforeRemoveLiquidity_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeRemoveLiquidity.lean#L49

P-49. `BaseHook.afterAddLiquidity`		
Status: **Verified**	Function: src/BaseHook.sol#L97	Proof: formal/Refinement/Swarm/AsyncSwapAfterAddLiquidity.lean

- `afterAddLiquidity_notPoolManager`: if the caller isn't `poolManager`, the call fails with `NotPoolManager`.
- `afterAddLiquidity_poolManager`: if the caller is `poolManager`, the call fails with `HookNotImplemented`. AsyncSwap doesn't override `_afterAddLiquidity`, so the base version runs and reverts.
- `afterAddLiquidity_spec`: the exact result is an if/else on the caller, giving one of those two errors.
- `afterAddLiquidity_never_ok`: no input or state gives a successful run. That covers the footprint: nothing is written, nothing is returned and no external calls are made.

Theorem	Status	Where
`afterAddLiquidity_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterAddLiquidity.lean#L20
`afterAddLiquidity_poolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterAddLiquidity.lean#L28
`afterAddLiquidity_spec`	Verified	formal/Refinement/Swarm/AsyncSwapAfterAddLiquidity.lean#L37
`afterAddLiquidity_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapAfterAddLiquidity.lean#L49

P-50. `BaseHook.afterRemoveLiquidity`		
Status: **Verified**	Function: src/BaseHook.sol#L120	Proof: formal/Refinement/Swarm/AsyncSwapAfterRemoveLiquidity.lean

This function can never succeed. AsyncSwap doesn't override `_afterRemoveLiquidity`, so once the caller check passes, the default version always reverts. The four theorems all name `BaseHook.afterRemoveLiquidity`: * `afterRemoveLiquidity_notPoolManager`: if `msg.sender ≠ poolManager`, it reverts with `NotPoolManager`. This is the one guard, at ImmutableState.sol:16. * `afterRemoveLiquidity_hookNotImplemented`: if the caller is `poolManager`, it reverts with `HookNotImplemented`. * `afterRemoveLiquidity_spec`: the exact result is always a revert, and which error it gives depends only on the caller. * `afterRemoveLiquidity_never_ok`: no call ever completes. So it

writes no storage, returns nothing and makes no external calls, which matches its empty footprint.

This isn't a bug. AsyncSwap's hook permissions set `afterRemoveLiquidity` to false (already proved in `AsyncSwapGetHookPermissions.lean`), so the pool manager never calls this function. I didn't need a PoC.

Theorem	Status	Where
`afterRemoveLiquidity_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterRemoveLiquidity.lean#L20
`afterRemoveLiquidity_hookNotImplemented`	Verified	formal/Refinement/Swarm/AsyncSwapAfterRemoveLiquidity.lean#L28
`afterRemoveLiquidity_spec`	Verified	formal/Refinement/Swarm/AsyncSwapAfterRemoveLiquidity.lean#L37
`afterRemoveLiquidity_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapAfterRemoveLiquidity.lean#L50

P-51. `BaseHook.afterSwap`		
Status: **Verified**	Function: src/BaseHook.sol#L160	Proof: formal/Refinement/Swarm/AsyncSwapAfterSwap.lean

- `afterSwap_notPoolManager`: if the caller isn't the pool manager, it reverts with `NotPoolManager` (the one guard, ImmutableState.sol:16).
- `afterSwap_hookNotImplemented`: if the caller is the pool manager, it reverts with `HookNotImplemented`, because AsyncSwap doesn't override `_afterSwap` (BaseHook.sol:170).
- `afterSwap_spec`: the exact behaviour: it always reverts, and which error depends only on who the caller is.
- `afterSwap_never_ok`: no call ever succeeds. So it writes no storage, returns nothing and makes no external calls, which matches the empty footprint in the slice.

Theorem	Status	Where
`afterSwap_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterSwap.lean#L20
`afterSwap_hookNotImplemented`	Verified	formal/Refinement/Swarm/AsyncSwapAfterSwap.lean#L28

Theorem	Status	Where
`afterSwap_spec`	Verified	formal/Refinement/Swarm/AsyncSwapAfterSwap.lean#L36
`afterSwap_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapAfterSwap.lean#L49

P-52. `BaseHook.beforeDonate`		
Status: **Verified**	Function: src/BaseHook.sol#L179	Proof: formal/Refinement/Swarm/AsyncSwapBeforeDonate.lean

All four theorems name `BaseHook.beforeDonate` fully qualified: * `beforeDonate_notPoolManager`: any caller other than the pool manager gets the `NotPoolManager` revert. * `beforeDonate_poolManager`: when the pool manager calls, it still reverts with `HookNotImplemented`. AsyncSwap doesn't override `_beforeDonate`, so the base version runs, and that one always reverts. * `beforeDonate_spec`: states exactly which of the two errors happens, depending on who the caller is. * `beforeDonate_never_ok`: no call ever succeeds. That proves the empty footprint (no storage writes, nothing returned, no external calls).

Theorem	Status	Where
`beforeDonate_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeDonate.lean#L20
`beforeDonate_poolManager`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeDonate.lean#L26
`beforeDonate_spec`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeDonate.lean#L33
`beforeDonate_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBeforeDonate.lean#L45

P-53. `BaseHook.afterDonate`		
Status: **Verified**	Function: src/BaseHook.sol#L198	Proof: formal/Refinement/Swarm/AsyncSwapAfterDonate.lean

- **Wrong caller:** `afterDonate_notPoolManager` shows that any caller other than `poolManager` makes it revert with `NotPoolManager`.

- **Pool manager calling:** `afterDonate_poolManager` shows it reverts with `HookNotImplemented`, because `_afterDonate` isn't overridden.
- **Full spec:** `afterDonate_spec` states the exact result, which is always one of those two reverts.
- **Never succeeds:** `afterDonate_never_ok` shows it never returns successfully. So on its own it writes no storage, makes no external calls and returns nothing, which matches the empty footprint in its slice.

This isn't a bug. AsyncSwap's hook permissions set `afterDonate := false`, so the pool manager never calls this hook, and the function always reverting is expected. I followed the same pattern as the existing `beforeDonate` proof.

Theorem	Status	Where
`afterDonate_notPoolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterDonate.lean#L20
`afterDonate_poolManager`	Verified	formal/Refinement/Swarm/AsyncSwapAfterDonate.lean#L26
`afterDonate_spec`	Verified	formal/Refinement/Swarm/AsyncSwapAfterDonate.lean#L33
`afterDonate_never_ok`	Verified	formal/Refinement/Swarm/AsyncSwapAfterDonate.lean#L45

4.4.3. AsyncSwap

P-13. `AsyncSwap.tryFillOrderFromPoolLiquidity`		
Status: **Verified**	Function: src/AsyncSwap.sol#L385	Proof: formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean

What the theorems show: * **Reverts only on its two guards.** It reverts exactly when `msg.sender` isn't an executor that `order.owner` authorised on the order's pool, or when the stored order has `amountIn = 0`. A failing fill doesn't make it revert, because the `try/catch` swallows it. * **Passes the checked order on unchanged.** When both guards pass, the result is exactly the self-call `this.fillOrderFromPoolLiquidity(order, id)`, made with the same `order` and the same `id` the guard just checked (the hash of chainid, pool id, owner, zeroForOne and salt). * **Success returns `true`; a failed fill changes nothing.** If the fill succeeds it returns `true` with the fill's resulting state. If the fill reverts it returns `false` and the state is exactly as before the call. * **Writes no storage of its own.** If the fill leaves

this contract's storage unchanged, so does this function.

For your concern about someone filling their own order to drain other users' funds: this wrapper can't be used to fill a different order than the one it checked, and a caller who isn't an authorised executor can't fill anyone's order through it. A failed fill can't leave partial changes behind either. What I haven't proved yet is whether the fill itself, `fillOrderFromPoolLiquidity`, can pay out more than the order escrowed. That's still open, and it's the next item to take.

I also added a small helper lemma (`run_tryCatch`) that evaluates `try/catch` in the model. The other `tryCatch` site in the generated model can reuse it.

Theorem	Status	Where
`tryFillOrderFromPoolLiquidity_eq`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L46
`tryFillOrderFromPoolLiquidity_notExecutor`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L72
`tryFillOrderFromPoolLiquidity_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L79
`tryFillOrderFromPoolLiquidity_pass`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L88
`tryFillOrderFromPoolLiquidity_ok`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L103
`tryFillOrderFromPoolLiquidity_reverts_iff`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L129
`tryFillOrderFromPoolLiquidity_false_noop`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L151
`tryFillOrderFromPoolLiquidity_storage`	Verified	formal/Refinement/Swarm/AsyncSwapTryFillOrderFromPoolLiquidity.lean#L160

P-15. `AsyncSwap.setExecutorWithSig`

Status: **Verified**	Function: src/AsyncSwap.sol#L533	Proof: formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean

Guards. Each theorem names `AsyncSwap.setExecutorWithSig` fully qualified and covers one outcome: * **Expired:** if the deadline has passed, it reverts with `SignatureExpired`. * **Nonce used:** if `nonceUsed[maker][nonce]` is already set, it reverts with `NonceAlreadyUsed`. * **Bad signature:** if the

signature check rejects it for `maker`, it reverts with `InvalidSignature`. * **Success:** otherwise the call burns the nonce and writes `asyncOrders[key.toId()].isExecutor[maker][executor] = allow`, and nothing more.

Who can change what (`_ok_inv`, `_ok_state`, `_frame`, `_no_replay`): * **Needs the maker's signature:** a call only succeeds if the signature check returned true for `maker`. The signed message covers exactly the call's arguments: key, maker, executor, allow, nonce and deadline. * **Writes only two things: It sets `nonceUsed[maker][nonce]`, which was false before the call.** It sets `maker's own executor flag in this one pool to `allow`. * **Leaves everything else alone:** other makers' executor flags, every pool's orders, every other pool, the price books, `owner`, `pendingOwner` and every other nonce stay the same. So this function can't move funds or give anyone authority over another user's orders. * **No replay:** once a call succeeds, any later call with the same `(maker, nonce)` reverts, whatever the other arguments or signature.

Assumptions. The last three properties rely on two things. First, `ext.get` returns what `ext.set` wrote. Second, three helpers the model leaves as black boxes don't change storage: the struct hash, `toTypedDataHash` and the ERC-1271 `staticcall`. In Solidity the first two are `pure` and the third is a `staticcall`, so this holds on chain, but the proof takes it as given. Inside the model I proved that the ECDSA recovery, `isValidSignatureNow` and `_hashTypedDataV4` don't change storage either, given those assumptions.

Theorem	Status	Where
`setExecutorWithSig_expired`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L94
`setExecutorWithSig_nonce_used`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L103
`setExecutorWithSig_invalid_sig`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L112
`setExecutorWithSig_ok`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L124
`setExecutorWithSig_ok_inv`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L139
`setExecutorWithSig_ok_state`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L187
`setExecutorWithSig_frame`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L206

Theorem	Status	Where
`setExecutorWithSig_no_replay`	Verified	formal/Refinement/Swarm/AsyncSwapSetExecutorWithSig.lean#L252

P-22. `AsyncSwap.cancelOrder`		
Status: **Verified**	Function: src/AsyncSwap.sol#L429	Proof: formal/Refinement/Swarm/AsyncSwapCancelOrder.lean, formal/Refinement/Swarm/Scratch.lean

What the theorems establish: * `cancelOrder_run`: the function's complete behaviour. It reverts with `NotAuthorizedExecutor` unless the caller is the order's owner or an executor the owner approved in that pool. It reverts with `OrderNotFound` if the stored order amount (`amountIn`) is 0. Otherwise it runs a fixed sequence: remove the order from the book, clear the order's storage slot, burn the hook's claim, then pay out and settle the priority fee. * `cancelOrder_unauthorized` **and** `cancelOrder_notFound`: each of those two reverts, stated on its own. * `cancelOrder_ok` (your concern about filling or cancelling your own order to drain other users): on success, the burn (from the hook itself) and the payout to the order's owner use the same amount in the same currency. That amount is the one in storage when the call starts, not the `amountIn` field the caller passes in. The order id is a hash that includes the owner, direction and pool, so a caller can only reach an order stored under their own address, or under an owner who approved them as executor, in the pool and currency it was created with. * `cancelOrder_reentry_reverts`: the order's slot is cleared before the first external call. So cancelling the same order again, including re-entering during the burn, always reverts and can't pay out twice.

Limits of these proofs: * The book-removal step is a black box in the generated model. That is where the third guard (`PriceLevelMissing`) and the `orderCount`/`openOrderCount` decrements happen, so none of that is proved here. * The external pool-manager calls are also black boxes. The theorems fix their arguments and order, not what they do. * The re-entry result assumes that writing our storage and reading it back returns what was written. * The same goes for the "owner-only" point: it assumes the hash used for the order id doesn't collide.

Theorem	Status	Where
`cancelOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L74
`cancelOrder_unauthorized`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L101

Theorem	Status	Where
`cancelOrder_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L108
`cancelOrder_ok`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L172
`cancelOrder_reentry_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L215

P-24. `AsyncSwap.cancelOrder`

Status: **Verified**	Function: src/AsyncSwap.sol#L429	Proof: formal/Refinement/Swarm/AsyncSwapCancelOrder.lean

Two parts of the model can't be seen into, so the proof doesn't cover them. The book-removal step (`_remove`), including its counter decrements and the `PriceLevelMissing` check, is treated as a black box. The model also doesn't say what the token manager's take-back and payout calls do inside, so I've only shown what they're called with. The re-entry proof also assumes that reading storage right after a write returns the written value.

Theorem	Status	Where
`cancelOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L74
`cancelOrder_unauthorized`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L101
`cancelOrder_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L108
`cancelOrder_ok`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L172
`cancelOrder_reentry_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapCancelOrder.lean#L215

P-25. `AsyncSwap.updatePrice`

Status: **Verified**	Function: src/AsyncSwap.sol#L418	Proof: formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean, formal/Scratch/UP.lean, formal/Scratch/UP2.lean, formal/Scratch/dummy.lean

What the proofs show: * **Exact behaviour:** `updatePrice_run` says the function does exactly this: it reverts with `NotOrderOwner` unless the caller is the order's owner, then with `ZeroAmount` if the new minimum output is 0, then with `OrderNotFound` if the stored order has no input. Otherwise it removes the order from the book, updates it, and adds it back. * **Each revert on its own:** `updatePrice_notOwner`, `updatePrice_zero` and `updatePrice_notFound`. * **Success case:** `updatePrice_ok` shows a successful call came from `order.owner` with a non-zero new minimum on an order that has input. Between removing and re-adding, the only changes are to that order's minimum output, price and sequence number, plus the pool book's next sequence number. The new price is always between 1 and 2^128, and the sequence counter can't overflow (it's checked against uint64). * **Nothing else changes:** the order's deposited input, owner, side and priority-fee fields stay the same, and so do every other order, every other pool, ownership, nonces and the book's open-order count and ordering mode.

Your self-fill / sweep concern: this function makes no external call and moves no tokens. It never changes the deposited `amountIn`. The order id is built from the owner's address, and only the owner may call it (not even an approved executor). So nobody can reprice someone else's order, and a reprice can't inflate what an order can later take out.

Limits: * **Book code not covered:** removing and re-adding the order happen in code the model treats as a black box. That's where the two price-tree checks from the function's slice (`leafId == 0`, `id == 0`) and the order-count changes live, and none of it is proved here. * **Storage assumption:** the success theorems assume that storage reads return what was last written.

Theorem	Status	Where
`updatePrice_run`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean#L74
`updatePrice_notOwner`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean#L94
`updatePrice_zero`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean#L100
`updatePrice_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean#L106

Theorem	Status	Where
`updatePrice_ok`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePrice.lean#L225

P-26. `AsyncSwap.recordPriorityFee`		
Status: **Verified**	Function: src/AsyncSwap.sol#L254	Proof: formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean, formal/Refinement/Swarm/Scratch.lean, formal/Refinement/Swarm/Scratch2.lean

The full behaviour, proved as one equation (`recordPriorityFee_run`). The call reverts in exactly three cases: . No match is pending (the pending order id is 0), or the caller isn't the pending match's executor. Both values come from transient storage, not from the call's arguments. . The stored order's owner isn't the `maker` passed in. . The order has no input left, its fee is already escrowed, `amount` differs from the order's stored `priorityFee`, or `amount` is 0.

Otherwise it does three things, in order: * mints exactly `amount` of the numeraire as claims to the protocol's own address; * sets that one order's `priorityFeeEscrowed` flag to true; * in priority-fee ordering mode only, removes the order from its price level and re-adds it.

What this means for your self-fill concern: * **Who can call it:** only the pending executor, only for the pending order, and only if that order really belongs to the named `maker` (`recordPriorityFee_unauthorized`, `recordPriorityFee_wrong_maker`). * **How much is credited:** exactly the order's stored fee, never a number the caller picks, and only to the protocol itself. No third party is paid and nothing leaves the protocol (`recordPriorityFee_amount_mismatch_reverts`). * **No double counting:** the flag write changes only that one order's flag. Other orders, other pools, the price books and transient storage are untouched (`flagWorld_frame`). Once the flag is set, any further call for that order reverts, whoever calls and whatever arguments they pass (`recordPriorityFee_escrowed_reverts`, `recordPriorityFee_twice_reverts`). * `recordPriorityFee_ok` gives all of the above as consequences of a successful call.

Limits: * In the generated model, the remove/re-add step and the PoolManager mint are opaque, so their internal effects (price-tree updates, order counters) are not covered here. * The frame and no-double-escrow results assume the model's storage accessor behaves normally (reading back a write returns what was written). * Within the model, an order's fee can be credited to the protocol only once, and only for the stored fee amount.

Next: the plan's next item.

Theorem	Status	Where
`recordPriorityFee_run`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L87
`recordPriorityFee_unauthorized`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L154
`recordPriorityFee_wrong_maker`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L160
`recordPriorityFee_escrowed_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L169
`recordPriorityFee_amount_mismatch_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L181
`recordPriorityFee_ok`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L197
`recordPriorityFee_twice_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapRecordPriorityFee.lean#L264

P-27. `AsyncSwap.executeOrder`

Status: **Verified**	Function: src/AsyncSwap.sol#L325	Proof: formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean, formal/Scratch.lean, formal/snippet.txt

What `executeOrder_run` (the full characterization) shows: . **Decoding `data`:** it reverts `abi.decode` if `data` is shorter than 32 bytes or its first word isn't an address. . **Native output only: If the `asyncOrderInfo` self-call reverts, that revert is passed through.** With `m` the minimum output that call returns, it panics if `m + calculatePoolFee(m)` overflows. **It reverts `UnaccountedNativeOutput` if the deposit recorded in this transaction (`pending`) is less than that total.** Otherwise it subtracts exactly that total from the deposit record. That write touches only its own transient slot (`tstoreWorld_frame`). . **Fee and guards:** it computes the fee from the stored minimum output, then runs `AsyncFiller.fill` with `hook = address(this)`, `executor = msg.sender`, the decoded `filler` and the fixed `feeVault`. It reverts `NotAuthorizedExecutor` or `OrderNotFound` when those checks fail (`fill_run`, `afterNative_run`).

On success (`executeOrder_ok`), the steps run in this order, using the stored input and minimum-output amounts, never the ones passed in `order`: * remove the order from the book; * for native output, settle `amountOutMin + fee` (checked add); * pay the minimum output to the owner; * if the fee is non-zero, mint it to the fee vault; * burn the input from the hook and pay it to the filler; * settle the priority fee; *

delete the order.

There is also a theorem for each separate revert path (`executeOrder_decode`, `_unaccountedNative`, `_nativeOverflow`, `_viewRevert`, `_unauthorized`, `_notFound`) and an exact formula for `calculatePoolFee`.

Limits: * The order-book removal is opaque in the generated model, so its `PriceLevelMissing` guard and counter decrements aren't covered. * The executor and not-found theorems are stated for ERC20 output only. Native output is covered by `executeOrder_run`, but not by stand-alone theorems.

Theorem	Status	Where
`executeOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L188
`executeOrder_decode`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L237
`executeOrder_unaccountedNative`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L245
`executeOrder_nativeOverflow`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L256
`executeOrder_viewRevert`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L266
`executeOrder_unauthorized`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L275
`executeOrder_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L290
`executeOrder_ok`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L323

P-28. `AsyncSwap.fillOrderFromPoolLiquidity`

Status: **Verified**	Function: src/AsyncSwap.sol#L401	Proof: formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean, formal/Refinement/Swarm/ScratchFill.lean, formal/Refinement/Swarm/ScratchFill2.lean

Guards (when it reverts): * `NotSelf`: if the caller isn't the contract itself. * `OrderNotFound`: if the stored order has `amountIn = 0`. * `NoActiveLiquidity`: if the pool's liquidity reads 0. In that case no swap is attempted. * `PartialLiquidityFill` **and** `LimitPriceNotMet`: these are proved from the success side. Any call that succeeds was a full fill at or above the limit (see below).

On success, the theorem gives the exact sequence of calls and what each is handed: * The escrowed input claims are burned from this contract: exactly the stored `amountIn`, in the input currency. * The pool's LP fee is set to 3000. * There is one exact-input swap with `-amountIn` and price limit MIN+1 or MAX-1. * The LP fee is put back to the value read from the pool before the swap. * The swap used exactly `amountIn`, `amountIn < 2^127`, the output is positive and `out ≥` the stored `amountOutMin`. * `out` of the output currency goes to the stored owner (`mint` if native, `take` otherwise), never a calldata field. * Then the order is unlinked from the book (`_remove`), `orders[id]` is reset to default, and the escrowed priority fee is settled.

Storage it writes (footprint): * The contract's own writes go only through the setters for `asyncOrders[pid]` and `priceBooks[pid]`, plus the direct reset of `orders[id]`. * The order slot is cleared before the priority-fee settlement makes its external calls, so re-entering to fill the same order from there reverts. * This needs the model's storage read-back law (`ext.get (ext.set w s) = s`) as a hypothesis.

Limits: the external calls and `_remove` are opaque in the generated model. So the guard on a missing price level (`PriceLevelMissing`) and the order-count decrements inside `_remove` aren't proved here. I also can't say anything about storage the PoolManager changes, including through re-entrant hook calls during the swap.

Theorem	Status	Where
`fillOrderFromPoolLiquidity_eq`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L76
`fillOrderFromPoolLiquidity_notSelf`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L120

Theorem	Status	Where
`fillOrderFromPoolLiquidity_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L126
`fillOrderFromPoolLiquidity_noLiquidity`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L133
`fillOrderFromPoolLiquidity_ok`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L261
`fillOrderFromPoolLiquidity_ok_full`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L293
`fillOrderFromPoolLiquidity_reentry_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapFillOrderFromPoolLiquidity.lean#L362

P-29. `AsyncSwap.updatePriceWithSig`

Status: **Verified**	Function: src/AsyncSwap.sol#L555	Proof: formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean, formal/Scratch.lean

What is proved: * **Exact unfolding** (`updatePriceWithSig_run`): the function equals `_verify` over the signed message, then the `ZeroAmount` check, then the `OrderNotFound` check, then the existing `tail` from the `updatePrice` proof (remove, write, append). The signed message covers owner, tokens, the stored `amountIn` read before verification, the new minimum output, order id, nonce and deadline. * **Each revert:** expired deadline (`SignatureExpired`), used nonce (`NonceAlreadyUsed`), rejected signature (`InvalidSignature`), zero new minimum (`ZeroAmount`), missing order (`OrderNotFound`). The not-the-owner check can never fire, because the function passes `order.owner` itself as the caller; the signature is the only authorization. * **A missing order or a zero minimum always reverts**, whatever the signature. * **Success** (`updatePriceWithSig_ok`): the deadline held, the nonce was fresh, and the signature check accepted the signature for `order.owner` over that exact message. Storage is then the nonce marked used, then remove, then the new minimum output, a price key in [1, 2^128) and a non-overflowing uint64 sequence, then append. The only external call is the ERC-1271 check, made with the owner, the digest and the signature. * **No replay** (`updatePriceWithSig_burns_nonce`): after success, `nonceUsed` is the old map with exactly (owner, nonce) set, and any later call with the same owner and nonce reverts.

What the proofs assume: * Writing storage and reading it back gives what was written. * The typed-data hash and the ERC-1271 check change no storage. The hash is `pure` and the check is a `staticcall`, but the model leaves both abstract. * For the no-replay theorem only: the remove and append steps, which the model also leaves abstract, don't change `nonceUsed`. * The two tree guards in the slice (`leafId == 0`, `id == 0`) sit inside remove and append, so these proofs don't cover them.

The next open item is `BaseHook.beforeSwap`.

Theorem	Status	Where
`updatePriceWithSig_run`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L79
`updatePriceWithSig_expired`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L108
`updatePriceWithSig_nonceUsed`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L114
`updatePriceWithSig_invalidSig`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L122
`updatePriceWithSig_zero`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L132
`updatePriceWithSig_notFound`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L139
`updatePriceWithSig_noOwnerGuard`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L150
`updatePriceWithSig_missing_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L219
`updatePriceWithSig_ok`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L242
`updatePriceWithSig_burns_nonce`	Verified	formal/Refinement/Swarm/AsyncSwapUpdatePriceWithSig.lean#L297

P-32. `AsyncSwap.matchBook`

Status: **Verified**	Function: src/AsyncSwap.sol#L412	Proof: formal/Scratch/MB.lean, formal/Refinement/Swarm/AsyncSwapMatchBook.lean, formal/Scratch/snippet.txt, formal/Refinement/Swarm/MBDebug.lean

What the theorems show: * **It only hands off to the library** (`matchBook_eq`): the function is exactly `AsyncMatching.matchBook` working on `asyncOrders[key.toId()]` and `priceBooks[key.toId()]`, with `hook = address(this)`. Same return value, same final state. * **Guard** (`matchBook_invalidBound`,

`matchBook_bound_noOk`): `maxMatches = 0` or `maxMatches > MAX_MATCHES_PER_CALL` (which is 8) reverts with `InvalidMatchBound` before any storage is touched. * **Return value** (`matchBook_ok`): on success, `1 ≤ maxMatches ≤ 8` and the returned match count is at most `maxMatches`. So the unchecked `uint8` counter `++matched` can never wrap. This comes from a loop invariant over the generated `for` loop. * **Footprint** (`selfSet_frame`, `bookSet_frame`): the storage setters handed to the library write only that one pool's `asyncOrders` or `priceBooks` slot. Every other pool, field, `owner`, `pendingOwner`, `nonceUsed` and transient storage stays the same. This holds as long as the model's `get` returns what `set` just stored.

Limits: * The other two guards in the slice (`leafId == 0` and the node-ID overflow) sit inside price-tree, `_remove` and `bestAtPrice` code that the model leaves opaque, so they aren't proved here. * What the external calls (`donate`, `burn`, `settle`, `take`, `mint`) do is whatever the model's assumptions say; nothing is proved about them. * `AsyncSwap.matchBook` has no caller check, unlike `fillOrderFromPoolLiquidity`. I didn't judge whether that is intended.

Theorem	Status	Where
`matchBook_eq`	Verified	formal/Refinement/Swarm/RouterMatchBook.lean#L27
`matchBook_invalidBound`	Verified	formal/Refinement/Swarm/AsyncSwapMatchBook.lean#L161
`matchBook_ok`	Verified	formal/Refinement/Swarm/AsyncSwapMatchBook.lean#L173
`matchBook_bound_noOk`	Verified	formal/Refinement/Swarm/AsyncSwapMatchBook.lean#L180

P-33. `AsyncSwap.matchNewOrder`

Status: **Verified**	Function: src/AsyncSwap.sol#L363	Proof: formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean, formal/Refinement/Swarm/_tail.lean.txt

The proofs are in `formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean`. Every theorem names `AsyncSwap.matchNewOrder` fully qualified. Most of them assume the model's state accessor is lawful (writing state and reading it back returns what was written).

- `matchNewOrder_run` **(full behaviour):** the function first clears both pending-match transient slots. Then:
  - it reverts if the pending id is 0 or the caller isn't the pending executor;

  - it reverts if the stored owner of order `id` isn't `order.owner`;
  - otherwise it runs `AsyncMatching.matchIncoming` on exactly `asyncOrders[pid]` and `priceBooks[pid]`, with `incomingId = id` and `hook = address(this)`;
  - if the order still has `amountIn ≠ 0`, it calls `this.fillOrderFromPoolLiquidity(order, id)`. If that call reverts, the revert is caught and the state goes back to how it was right after matching;
  - it returns the count from `matchIncoming`.
- **Guards:** `matchNewOrder_unauthorized`, `matchNewOrder_wrongOwner` and `matchNewOrder_emptyOrder` (an order with `amountIn = 0` hits `matchIncoming`'s guard and reverts). The two `RadixPriceTree` guards sit inside tree calls the model leaves opaque, so they aren't proved here, the same as for `matchBook`.
- **Success (`matchNewOrder_ok`):** a successful call had the pending executor as caller, a pending id that isn't 0 and the right owner. It returns at most 8 matches (`matchIncoming_le`), so the unchecked `uint8 ++matched` never wraps.
- **Footprint and re-entry:** `clearWorld_frame` shows the clear zeroes only the two pending-match slots and changes nothing else. `matchNewOrder_reentry_reverts` shows any re-entrant call made after the clear reverts. The `asyncOrders` and `priceBooks` writes reuse `selfSet_frame` and `bookSet_frame` from the `matchBook` proofs, which this file imports.

Theorem	Status	Where
`matchNewOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L74
`matchNewOrder_unauthorized`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L140
`matchNewOrder_wrongOwner`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L169
`matchNewOrder_emptyOrder`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L191
`matchNewOrder_reentry_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L207
`matchNewOrder_ok`	Verified	formal/Refinement/Swarm/AsyncSwapMatchNewOrder.lean#L252

P-34. `AsyncSwap.getHookPermissions`

Status: **Verified**	Function: src/AsyncSwap.sol#L193	Proof: formal/Refinement/Swarm/AsyncSwapGetHookPermissions.lean

- **Full behaviour:** `getHookPermissions_run` shows that in any environment and storage state, the call succeeds, leaves storage unchanged and returns a fixed permission set. The proof is `rfl`, i.e. it holds by unfolding the definition.
- **No guards:** `getHookPermissions_never_reverts` shows it can never revert.
- **No writes:** `getHookPermissions_ok_iff` shows a successful result means exactly that fixed set and unchanged storage.
- **Returned flags:** `getHookPermissions_fields` checks every flag. Only `beforeInitialize`, `beforeAddLiquidity`, `beforeSwap` and `beforeSwapReturnDelta` are true; the other ten are false. This matches the Solidity at `src/AsyncSwap.sol:193`.

Theorem	Status	Where
`getHookPermissions_run`	Verified	formal/Refinement/Swarm/AsyncSwapGetHookPermissions.lean#L29
`getHookPermissions_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapGetHookPermissions.lean#L33
`getHookPermissions_ok_iff`	Verified	formal/Refinement/Swarm/AsyncSwapGetHookPermissions.lean#L39
`getHookPermissions_fields`	Verified	formal/Refinement/Swarm/AsyncSwapGetHookPermissions.lean#L47

P-35. `AsyncSwap.asyncOrder`

Status: **Verified**	Function: src/AsyncSwap.sol#L224	Proof: formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean

The proof is in `formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean`. Each theorem names `AsyncSwap.asyncOrder` in its statement: * `asyncOrder_run`: for any environment and world, the function succeeds and returns `asyncOrders[poolId].orders[orderId].amountIn`. The world comes back unchanged. * `asyncOrder_never_reverts`: it has no guards and never reverts. * `asyncOrder_ok_iff`: it succeeds with result `a` and world `w'` exactly when `a` is that stored `amountIn` and `w'` is the starting world, so it writes nothing. * `asyncOrder_reads_only_asyncOrders`: two worlds that agree on `asyncOrders` give the same result, whatever the environment. So `asyncOrders` is the only storage it reads.

It makes no external calls.

Theorem	Status	Where
`asyncOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean#L20
`asyncOrder_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean#L25
`asyncOrder_ok_iff`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean#L31
`asyncOrder_reads_only_asyncOrders`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrder.lean#L41

P-36. `AsyncSwap.asyncOrderInfo`

Status: **Verified**	Function: src/AsyncSwap.sol#L229	Proof: formal/Refinement/Swarm/AsyncSwapAsyncOrderInfo.lean

The function has no guards, which matches its slice. Every theorem names `AsyncSwap.asyncOrderInfo` fully qualified: * `asyncOrderInfo_run`: in any environment and storage state, the call succeeds, leaves storage unchanged and returns `(amountIn, amountOutMin)` for `asyncOrders[poolId].orders[orderId]`. Lean accepts this by direct computation on the def (`rfl`). * `asyncOrderInfo_never_reverts`: the call never reverts. * `asyncOrderInfo_ok_iff`: it succeeds with exactly that pair, and the storage afterwards equals the storage before, so it writes nothing. * `asyncOrderInfo_reads_only_asyncOrders`: the result depends only on `asyncOrders`. Two storage states that agree there give the same answer, whatever the environment.

It makes no external calls. The file follows the existing `AsyncSwapAsyncOrder.lean` proof.

Theorem	Status	Where
`asyncOrderInfo_run`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderInfo.lean#L21
`asyncOrderInfo_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderInfo.lean#L27
`asyncOrderInfo_ok_iff`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderInfo.lean#L33

Theorem	Status	Where
`asyncOrderInfo_reads_only_asyncOrders`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderInfo.lean#L44

P-37. `AsyncSwap.asyncOrderPriorityFee`

Status: **Verified**	Function: src/AsyncSwap.sol#L239	Proof: formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean

The main theorem, `asyncOrderPriorityFee_run`, pins down what `AsyncSwap.asyncOrderPriorityFee` does in every environment and world: * **Reverts:** never. It has no guards. * **Writes:** nothing. The world is returned unchanged. * **External calls:** none. * **Returns:** `info.priorityFee` if `info.priorityFeeEscrowed` is set, otherwise 0, where `info` is `asyncOrders[poolId].orders[orderId]`. This matches `AsyncFiller.paidPriorityFee`.

The other theorems follow from it: * `asyncOrderPriorityFee_never_reverts`: it never returns an error. * `asyncOrderPriorityFee_ok_iff`: a success result means exactly the value above with an unchanged world. * `asyncOrderPriorityFee_escrowed` and `asyncOrderPriorityFee_not_escrowed`: one theorem for each value of the escrow flag. * `asyncOrderPriorityFee_reads_only_asyncOrders`: two worlds that agree on `asyncOrders` give the same answer, so it reads nothing else.

The property holds.

Theorem	Status	Where
`asyncOrderPriorityFee_run`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L21
`asyncOrderPriorityFee_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L30
`asyncOrderPriorityFee_ok_iff`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L36
`asyncOrderPriorityFee_not_escrowed`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L46
`asyncOrderPriorityFee_escrowed`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L52
`asyncOrderPriorityFee_reads_only_asyncOrders`	Verified	formal/Refinement/Swarm/AsyncSwapAsyncOrderPriorityFee.lean#L59

P-38. `AsyncSwap.orderingMode`		
Status: **Verified**	Function: src/AsyncSwap.sol#L243	Proof: formal/Refinement/Swarm/AsyncSwapOrderingMode.lean

The function has no guards and makes no external calls, so there are four short theorems: * `orderingMode_run`: the exact result. For any environment and world, the call returns `priceBooks[poolId].orderingMode` from the hook's storage and leaves the world unchanged. * `orderingMode_never_reverts`: it never fails with any revert reason. * `orderingMode_frame`: when it succeeds, the world afterwards is the same as before, so it writes nothing. * `orderingMode_result`: when it succeeds, the value returned is the stored ordering mode of that pool's price book.

Theorem	Status	Where
`orderingMode_run`	Verified	formal/Refinement/Swarm/AsyncSwapOrderingMode.lean#L18
`orderingMode_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapOrderingMode.lean#L24
`orderingMode_frame`	Verified	formal/Refinement/Swarm/AsyncSwapOrderingMode.lean#L29
`orderingMode_result`	Verified	formal/Refinement/Swarm/AsyncSwapOrderingMode.lean#L34

P-39. `AsyncSwap.bestOrder`		
Status: **Verified**	Function: src/AsyncSwap.sol#L271	Proof: formal/Refinement/Swarm/Scratch39.lean, formal/Refinement/Swarm/AsyncSwapBestOrder.lean

The proof covers a side of the book `T`, which is `priceBooks[poolId].bids` when `isBuy` is true and `.asks` otherwise: * **Full characterization (`bestOrder_run`):** `AsyncSwap.bestOrder` is described exactly. **If `T.root = 0`, it returns `(false, 0, 0, 0)` and changes no state.** Otherwise it walks down from the root, going right on bids and left on asks, to the first leaf `c`. **If no leaf turns up within the model's `loopBound` of 2^64 steps, it reverts with `"loop bound"`. That limit stands in for running out of gas, and it's the only revert in this function's own code.** Once a leaf is found, it returns `(true, p, orderId, maker)`, where `p` is the leaf's price, `orderId` is what `bestAtPrice(book, isBuy, p)` returns, and `maker` is the owner stored for that order. * **Corollaries:** `bestOrder_empty` covers the empty side and `bestOrder_loopBound` the

revert. `bestOrder_ok` gives the success shape: the leaf is reached from the root in fewer than 2^64 steps, every node before it is a branch, and the result is exactly the values above. * **Footprint:** the tree walk (`best_run`, which works for any read-only accessor) never writes storage. `bestAtPrice` is treated as an opaque step in the generated model, so `bestOrder_readOnly` shows the whole call changes nothing only under the assumption that `bestAtPrice` is read-only. That matches Solidity, where it is a `view`, but the theorem relies on it rather than proving it.

Theorem	Status	Where
`bestOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapBestOrder.lean#L155
`bestOrder_empty`	Verified	formal/Refinement/Swarm/AsyncSwapBestOrder.lean#L202
`bestOrder_loopBound`	Verified	formal/Refinement/Swarm/AsyncSwapBestOrder.lean#L208
`bestOrder_ok`	Verified	formal/Refinement/Swarm/AsyncSwapBestOrder.lean#L218
`bestOrder_readOnly`	Verified	formal/Refinement/Swarm/AsyncSwapBestOrder.lean#L251

P-40. `AsyncSwap.orderBookLevel`

Status: **Verified**	Function: src/AsyncSwap.sol#L280	Proof: formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean, formal/Refinement/Swarm/AsyncSwapOrderBookLevel.head.tmp

I also left a stray empty file, `formal/Refinement/Swarm/AsyncSwapOrderBookLevel.head.tmp`, which my tools can't delete; it's harmless and can be removed by hand. Next: the swarm re-runs the build and coverage before review. The opaque `AsyncFiller.level` (FIFO head and tail, or the best priority keys) stays unproved until formal-ast generates a def for it.

Theorem	Status	Where
`orderBookLevel_eq`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L41
`orderBookLevel_run`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L48

Theorem	Status	Where
`orderBookLevel_reverts_iff`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L55
`orderBookLevel_readOnly`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L63
`orderBookLevel_view`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L73
`orderBookLevel_congr`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookLevel.lean#L86

P-41. `AsyncSwap.orderBookOrder`

Status: **Verified**	Function: src/AsyncSwap.sol#L289	Proof: formal/Refinement/Swarm/AsyncSwapOrderBookOrder.lean

It has four theorems: * `orderBookOrder_run` **(return value):** the function always succeeds and returns `owner, amountIn, amountOutMin, price, isBuy, previous, next` from `asyncOrders[poolId].orders[orderId]`. The state comes back unchanged. * `orderBookOrder_never_reverts` **(guards):** it has no guards and never reverts. * `orderBookOrder_readOnly` **(footprint):** it writes no storage. * `orderBookOrder_congr`**:** the result depends only on that one order record.

Theorem	Status	Where
`orderBookOrder_run`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookOrder.lean#L25
`orderBookOrder_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookOrder.lean#L32
`orderBookOrder_readOnly`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookOrder.lean#L37
`orderBookOrder_congr`	Verified	formal/Refinement/Swarm/AsyncSwapOrderBookOrder.lean#L44

P-42. `AsyncSwap.calculateHookFee`

Status: **Verified**	Function: src/AsyncSwap.sol#L310	Proof: formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean

I proved plan item 42: `AsyncSwap.calculateHookFee` matches its slice. The proof is in `formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean`, and `lake build Refinement.Swarm.AsyncSwapCalculateHookFee` succeeds with no `sorry`, `admit`, `native_decide` or new axioms. The only build warnings are about unreachable code in `Generated/AsyncSwap.lean`, not in the new file.

Theorem	Status	Where
`calculateHookFee_run`	Verified	formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean#L20
`calculateHookFee_never_reverts`	Verified	formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean#L24
`calculateHookFee_ok_iff`	Verified	formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean#L30
`calculateHookFee_const`	Verified	formal/Refinement/Swarm/AsyncSwapCalculateHookFee.lean#L38

P-43. `AsyncSwap.DOMAIN_SEPARATOR`

Status: **Verified**	Function: src/AsyncSwap.sol#L584	Proof: formal/Refinement/Swarm/AsyncSwapDOMAIN_SEPARATOR.lean

The proof is in `formal/Refinement/Swarm/AsyncSwapDOMAIN_SEPARATOR.lean` and has three theorems about `AsyncSwap.DOMAIN_SEPARATOR`:

- `DOMAIN_SEPARATOR_spec`: for any environment and state, the call succeeds and leaves the state unchanged. It returns the cached separator when `address(this)` and `block.chainid` match the cached values. Otherwise it returns `keccak[TYPE_HASH, _hashedName, _hashedVersion, chainid, self]`, the separator rebuilt from scratch.
- `DOMAIN_SEPARATOR_never_reverts`: the function has no guards, so it can never revert.
- `DOMAIN_SEPARATOR_preserves`: if the call succeeds, storage is unchanged and the result is that separator.

Theorem	Status	Where
DOMAIN_SEPARATOR_spec	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L29
DOMAIN_SEPARATOR_never_reverts	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L40
DOMAIN_SEPARATOR_preserves	Verified	formal/Refinement/Swarm/RouterDOMAIN_SEPARATOR.lean#L45

P-44. AsyncSwap.«_1952»

Status: **Verified**	Function: src/AsyncSwap.sol#L606	Proof: formal/Refinement/Swarm/AsyncSwapReceive.lean

- receive_run: for any environment and world, receive succeeds, returns (), and the only change is transient[PENDING_NATIVE_DEPOSIT_SLOT] := yadd pending msg.value. This holds by rfl.
- receive_never_reverts: it has no guards, so it never reverts.
- receive_ok: any successful run ends in exactly that world.
- receive_frame: this one assumes ext is lawful, meaning reading back after a write returns what was written. Under that assumption, the ledger slot becomes (pending + msg.value) mod 2^256. Every declared storage field (asyncOrders, priceBooks, owner, pendingOwner, nonceUsed, and the two name/version fallback fields) and every other transient slot stays the same. It makes no external calls.

P-54. AsyncSwap.calculatePoolFee

Status: **Verified**	Function: src/AsyncSwap.sol#L315	Proof: formal/Refinement/Swarm/AsyncSwapCalculatePoolFee.lean

The proofs are in formal/Refinement/Swarm/AsyncSwapCalculatePoolFee.lean, and each theorem names AsyncSwap.calculatePoolFee in full: * calculatePoolFee_run: gives the exact behaviour for any input. It fails with panic 0x11 (arithmetic overflow) only if the source expression's result would overflow. Otherwise it returns that result and leaves the stored state unchanged. * calculatePoolFee_ok: for any amount < 2^256 (every valid uint256), the function never reverts, writes nothing and returns exactly amount * 3 / 997. So splitting the calculation in two loses no precision compared with the plain formula. * calculatePoolFee_le: the returned fee is never more than amount, and the state comes back unchanged.

Theorem	Status	Where
`calculatePoolFee_run`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L124
`calculatePoolFee_ok`	Verified	formal/Refinement/Swarm/AsyncSwapCalculatePoolFee.lean#L46
`calculatePoolFee_le`	Verified	formal/Refinement/Swarm/AsyncSwapCalculatePoolFee.lean#L53
`calculatePoolFee_run`	Verified	formal/Refinement/Swarm/AsyncSwapExecuteOrder.lean#L124

4.4.4. NativeClaims

P-14. `NativeClaims.redeem`		
Status: **Verified**	Function: src/NativeClaims.sol#L30	Proof: formal/Refinement/Swarm/NativeClaimsRedeem.lean

What the theorems show: * **Exact behaviour (`redeem_run`): It reverts with `ReentrantRedemption` if `_redeeming` is already set.** Otherwise it reverts with `InvalidRedemption` if the recipient or the amount is 0. **Otherwise it calls `poolManager.unlock(abiEncode [msg.sender, recipient, amount])` with `_redeeming = true`. A revert from that call comes back to the caller unchanged. On success the result is the call's final state with `_redeeming` cleared.** * The two guards on their own: `redeem_reentrant` **and** `redeem_invalid`**.** * When it succeeds (`redeem_ok`): **both guards passed, and the owner word sent to the pool manager is always `msg.sender`. No argument lets a caller name someone else as owner.** * When it reverts (`redeem_reverts_iff`): **exactly when one of the guards fails or the pool-manager call reverts.** * Storage written (`redeem_ok_state`): **this assumes a well-behaved storage accessor, i.e. reading back what was just written gives the same value. Under that, the external call sees `_redeeming = true`, and it is `false` again after a successful run.** * Re-entry (`redeem_nested_reverts`):** any nested `redeem` made during the call reverts with `ReentrantRedemption`.

Theorem	Status	Where
`redeem_run`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L29
`redeem_reentrant`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L53

Theorem	Status	Where
`redeem_invalid`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L59
`redeem_ok`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L67
`redeem_reverts_iff`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L88
`redeem_ok_state`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L109
`redeem_nested_reverts`	Verified	formal/Refinement/Swarm/NativeClaimsRedeem.lean#L117

P-16. `NativeClaims.unlockCallback`

Status: **Verified**	Function: src/NativeClaims.sol#L35	Proof: formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean, /tmp/ignore.txt

I proved plan item 16: `NativeClaims.unlockCallback` matches its slice. The proofs are in `formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean`, and `lake build Refinement.Swarm.NativeClaimsUnlockCallback` passes with no `sorry`, `admit`, `native_decide` or new axioms.

Theorem	Status	Where
`unlockCallback_run`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L46
`unlockCallback_onlyPoolManager`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L70
`unlockCallback_badData`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L76
`unlockCallback_ok`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L85
`unlockCallback_burn_fails`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L112

Theorem	Status	Where
`unlockCallback_reverts_iff`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L119
`unlockCallback_ok_args`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L154

4.4.5. FeeVault

P-17. `FeeVault.collect`		
Status: **Verified**	Function: src/FeeVault.sol#L27	Proof: formal/Refinement/Swarm/FeeVaultCollect.lean

- **Full run (`collect_run`):** the function first asks the hook for its owner, and if that read fails, the function fails the same way. Otherwise:
  - a caller who isn't the owner gets `Unauthorized`;
  - a zero recipient gets `ZeroAddress`;
  - otherwise it calls `poolManager.unlock(abi.encode(currency, recipient))` on the unchanged state, and the final state is whatever that call leaves.
- **Footprint:** FeeVault has no storage of its own, so the pool-manager call is its only effect.
- **Each guard on its own:** `collect_owner_view_reverts`, `collect_unauthorized` and `collect_zero_recipient`.
- **Success (`collect_ok`):** a successful call means the caller is the owner, the recipient is nonzero, and the pool manager got exactly `abi.encode(currency, recipient)`.
- **Revert conditions (`collect_reverts_iff`):** the function reverts exactly when one of those four steps fails.
- **Access control (`collect_only_owner`):** every successful call came from `hook.owner()`.

Theorem	Status	Where
`collect_run`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L31
`collect_owner_view_reverts`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L58

Theorem	Status	Where
`collect_unauthorized`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L64
`collect_zero_recipient`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L70
`collect_ok`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L78
`collect_only_owner`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L101
`collect_reverts_iff`	Verified	formal/Refinement/Swarm/FeeVaultCollect.lean#L108

P-18. `FeeVault.unlockCallback`		
Status: **Verified**	Function: src/FeeVault.sol#L34	Proof: formal/Refinement/Swarm/FeeVaultUnlockCallback.lean, formal/.scratch_ignore

- **What a call does:** `unlockCallback_run` describes a call step by step:
  - It reverts `Unauthorized` unless the caller is the pool manager.
  - It reverts `abi.decode` unless `data` has at least 64 bytes and both words fit in an address.
  - It then reads the vault's own claim balance for that currency. If that read reverts, the callback reverts with the same reason.
  - If the balance is 0, it changes nothing and returns `""`.
  - Otherwise it runs `burn(address(this), currency, amount)` and then `take(currency, recipient, amount)`. Either revert passes straight through, and the vault writes no storage of its own.
- **Guard and revert theorems:** separate theorems cover the wrong caller, bad data, a failing balance read, the zero-balance case, the non-zero case, and an exact list of when it reverts.
- **On success (`unlockCallback_ok`):** the return value is `""`. The final world is either unchanged (balance was 0) or exactly the one `burn` then `take` leave, using the same `amount`.
- **Your concern (draining via a crafted order):** `unlockCallback_take_eq_own_balance` shows that the amount sent out is exactly the vault's own fee balance for that currency, read just before. That amount is first burned from the vault's own account, never from another user's claim.

`unlockCallback_burn_fails` shows nothing is taken out if the burn fails. Only the pool manager can call this function.

The proofs assume nothing about how the pool manager's `burn`, `take` and `balanceOf` behave; they are left open. So these results say what the vault asks the pool manager to do, not how the pool manager carries it out.

Theorem	Status	Where
`unlockCallback_run`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L46
`unlockCallback_unauthorized`	Verified	formal/Refinement/Swarm/FeeVaultUnlockCallback.lean#L86
`unlockCallback_badData`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L76
`unlockCallback_view_reverts`	Verified	formal/Refinement/Swarm/FeeVaultUnlockCallback.lean#L98
`unlockCallback_zero`	Verified	formal/Refinement/Swarm/FeeVaultUnlockCallback.lean#L105
`unlockCallback_nonzero`	Verified	formal/Refinement/Swarm/FeeVaultUnlockCallback.lean#L113
`unlockCallback_ok`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L85
`unlockCallback_take_eq_own_balance`	Verified	formal/Refinement/Swarm/FeeVaultUnlockCallback.lean#L165
`unlockCallback_burn_fails`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L112
`unlockCallback_reverts_iff`	Verified	formal/Refinement/Swarm/NativeClaimsUnlockCallback.lean#L119

Appendix A: Function Coverage

Table 2. AsyncSwap

Function	Status	Theorems
AsyncSwap.constructor	Not modeled	0
AsyncSwap.onlyOwner	Proved	8
AsyncSwap.transferOwnership	Proved	6
AsyncSwap.acceptOwnership	Proved	5
CurrencyLibrary.isAddressZero	Proved	14
CurrencyLibrary.balanceOfSelf	Proved	3
CustomRevert.bubbleUpAndRevertWith	In Lean	0
CurrencyLibrary.transfer	Not modeled	0
AsyncSwap.sweep	Proved	6
AsyncSwap.getHookPermissions	Proved	4
AsyncSwap.asyncOrder	Proved	4
AsyncSwap.asyncOrderInfo	Proved	4
AsyncFiller.paidPriorityFee	Proved	3
AsyncSwap.asyncOrderPriorityFee	Proved	6
AsyncSwap.orderingMode	Proved	4
PoolIdLibrary.toId	Proved	26
AsyncFiller.setOrderingMode	Proved	2
AsyncMatching.setOrderingMode	Proved	3
AsyncSwap.setOrderingMode	Proved	5
CurrencyLibrary.toId	Proved	7
CurrencySettler.take	Proved	5
RadixPriceTree._freeNode	In Lean	0
RadixPriceTree.remove	In Lean	0

Function	Status	Theorems
AsyncFiller._remove	Not modeled	0
RadixPriceTree._newNode	In Lean	0
RadixPriceTree._bitSet	In Lean	0
RadixPriceTree._mostSignificantBit	In Lean	0
RadixPriceTree.insert	In Lean	0
AsyncFiller._priorityKey	In Lean	0
AsyncFiller._append	Not modeled	0
AsyncFiller.recordPriorityFee	Proved	1
AsyncMatching.recordPriorityFee	Proved	1
AsyncSwap.recordPriorityFee	Proved	7
RadixPriceTree.best	Proved	1
AsyncFiller.bestAtPrice	Not modeled	0
AsyncFiller.bestOrder	Proved	1
AsyncMatching.bestOrder	Proved	1
AsyncSwap.bestOrder	Proved	5
AsyncFiller.level	Not modeled	0
AsyncMatching.level	Proved	1
AsyncSwap.orderBookLevel	Proved	6
AsyncSwap.orderBookOrder	Proved	4
AsyncSwap.isExecutor	Proved	1
AsyncSwap.calculateHookFee	Proved	4
AsyncSwap.calculatePoolFee	Proved	4
AsyncOrderLibrary.orderId	Proved	5
Address._revert	In Lean	0
Address.verifyCallResultFromTarget	In Lean	0

Function	Status	Theorems
Address.functionCallWithValue	In Lean	0
Address.functionCall	In Lean	0
SafeERC20._callOptionalReturn	In Lean	0
SafeERC20.safeTransferFrom	In Lean	0
CurrencySettler.settle	Proved	3
StateLibrary._getPoolStateSlot	Not modeled	0
StateLibrary.getLiquidity	Proved	3
AsyncFiller.settlePriorityFee	Proved	4
AsyncFiller.settleOrderPriorityFee	In Lean	0
AsyncFiller.fill	Proved	1
AsyncMatching.fill	Proved	1
AsyncSwap.executeOrder	Proved	8
Math.mulDiv	In Lean	0
AsyncMatching._crosses	In Lean	0
Math.unsignedRoundsUp	In Lean	0
Math.mulDiv	In Lean	0
AsyncMatching._matchAmounts	In Lean	0
AsyncMatching._transferEscrow	In Lean	0
AsyncMatching._settleTransfers	In Lean	0
AsyncMatching._settleMatchedPriorityFee	In Lean	0
AsyncMatching._priceKey	In Lean	0
AsyncMatching._reduceAfterMatch	In Lean	0
AsyncMatching._reduceMatchedOrders	In Lean	0
AsyncMatching._matchPair	In Lean	0
AsyncMatching.matchIncoming	Proved	3

Function	Status	Theorems
AsyncSwap.matchNewOrder	Proved	6
AsyncSwap.tryFillOrderFromPoolLiquidity	Proved	8
StateLibrary.getSlot0	Proved	2
CustomRevert.revertWith	Proved	1
SafeCast.toInt256	Proved	1
BalanceDeltaLibrary.amount1	Proved	1
BalanceDeltaLibrary.amount0	Proved	1
SafeCast.toInt128	Proved	1
AsyncMatching._swapAgainstPool	In Lean	0
AsyncMatching.fillFromPoolLiquidity	Proved	2
AsyncSwap.fillOrderFromPoolLiquidity	Proved	7
AsyncMatching.matchBook	Proved	3
AsyncSwap.matchBook	Proved	4
AsyncFiller._priceKey	Proved	2
AsyncFiller.updatePrice	Proved	2
AsyncMatching.updatePrice	Proved	1
AsyncSwap.updatePrice	Proved	5
AsyncFiller.cancel	Proved	1
AsyncMatching.cancel	Proved	1
AsyncSwap.cancelOrder	Proved	5
AsyncSwap.setExecutor	Proved	9
ECDSA.tryRecover	Proved	2
ECDSA.tryRecover	Proved	2
SignatureChecker.isValidERC1271SignatureNow	Not modeled	0
SignatureChecker.isValidSignatureNow	Proved	13

Function	Status	Theorems
MessageHashUtils.toTypedDataHash	Not modeled	0
EIP712._buildDomainSeparator	Proved	4
EIP712._domainSeparatorV4	Proved	4
EIP712._hashTypedDataV4	Proved	13
AsyncSwap._verify	Proved	13
OrderSignatures.hashPoolKey	Not modeled	0
OrderSignatures.hashSetExecutor	Not modeled	0
AsyncSwap.setExecutorWithSig	Proved	8
OrderSignatures.tokens	Proved	2
OrderSignatures.hashUpdatePrice	Proved	1
AsyncSwap.updatePriceWithSig	Proved	10
AsyncSwap.invalidateNonce	Proved	6
AsyncSwap.DOMAIN_SEPARATOR	Proved	3
AsyncSwap.receive	Proved	4
ShortStrings.byteLength	In Lean	0
ShortStrings.toString	Not modeled	0
ShortStrings.toStringWithFallback	Not modeled	0
EIP712._EIP712Name	In Lean	0
EIP712._EIP712Version	In Lean	0
EIP712.eip712Domain	Not modeled	0
ImmutableState.onlyPoolManager	Proved	26
AsyncSwap._beforeInitialize	Proved	3
BaseHook.beforeInitialize	Proved	8
BaseHook._afterInitialize	Proved	1
BaseHook.afterInitialize	Proved	4

Function	Status	Theorems
AsyncSwap._beforeAddLiquidity	Proved	1
BaseHook.beforeAddLiquidity	Proved	4
BaseHook._beforeRemoveLiquidity	Proved	1
BaseHook.beforeRemoveLiquidity	Proved	4
BaseHook._afterAddLiquidity	Proved	1
BaseHook.afterAddLiquidity	Proved	4
BaseHook._afterRemoveLiquidity	Proved	1
BaseHook.afterRemoveLiquidity	Proved	4
AsyncFiller.addOrder	In Lean	0
AsyncMatching.addOrder	In Lean	0
AsyncSwap._createOrder	Not modeled	0
?.toBeforeSwapDelta	Proved	1
AsyncSwap._beforeSwap	Proved	5
BaseHook.beforeSwap	Proved	11
BaseHook._afterSwap	Proved	1
BaseHook.afterSwap	Proved	4
BaseHook._beforeDonate	Proved	1
BaseHook.beforeDonate	Proved	4
BaseHook._afterDonate	Proved	1
BaseHook.afterDonate	Proved	4

Table 3. FeeVault

Function	Status	Theorems
FeeVault.constructor	Not modeled	0
FeeVault.collect	Proved	7
CurrencyLibrary.toId	Proved	7

Function	Status	Theorems
FeeVault.unlockCallback	Proved	10

Table 4. NativeClaims

Function	Status	Theorems
NativeClaims.constructor	Not modeled	0
NativeClaims.nonReentrant	Proved	1
NativeClaims.redeem	Proved	7
NativeClaims.unlockCallback	Proved	7

Table 5. Router

Function	Status	Theorems
Router.constructor	Not modeled	0
CurrencyLibrary.isAddressZero	Proved	14
Router.swap	Proved	2
PoolIdLibrary.toId	Proved	26
Router._orderId	Proved	2
Router.fillOrder	Proved	3
Router.swapWithSig	Proved	6
MessageHashUtils.toTypedDataHash	Not modeled	0
EIP712._buildDomainSeparator	Proved	4
EIP712._domainSeparatorV4	Proved	4
EIP712._hashTypedDataV4	Proved	13
OrderSignatures.tokens	Proved	2
OrderSignatures.hashCancel	Proved	1
ECDSA.tryRecover	Proved	2
ECDSA.tryRecover	Proved	2

Function	Status	Theorems
SignatureChecker.isValidERC1271SignatureNow	Not modeled	0
SignatureChecker.isValidSignatureNow	Proved	13
Router.cancelWithSig	Proved	10
Router.DOMAIN_SEPARATOR	Proved	3
Router.cancelOrder	Proved	2
Router.matchBook	Proved	3
Router.fillFromPoolLiquidity	Proved	3
Router.onlyPoolManager	Proved	2
Address._revert	In Lean	0
Address.verifyCallResultFromTarget	In Lean	0
Address.functionCallWithValue	In Lean	0
Address.functionCall	In Lean	0
SafeERC20._callOptionalReturn	In Lean	0
SafeERC20.safeTransferFrom	In Lean	0
CustomRevert.revertWith	Proved	1
SafeCast.toInt256	Proved	1
CustomRevert.bubbleUpAndRevertWith	In Lean	0
CurrencyLibrary.transfer	Not modeled	0
CurrencyLibrary.toId	Proved	7
CurrencySettler.settle	Proved	3
OrderSignatures.hashPoolKey	Not modeled	0
OrderSignatures.hashOrder	Not modeled	0
Router._settleWithPermit	Not modeled	0
Router._settlePriorityFee	Not modeled	0
Router.unlockCallback	Proved	3

Function	Status	Theorems
ShortStrings.byteLength	In Lean	0
ShortStrings.toString	Not modeled	0
ShortStrings.toStringWithFallback	Not modeled	0
EIP712._EIP712Name	In Lean	0
EIP712._EIP712Version	In Lean	0
EIP712.eip712Domain	Not modeled	0